

Informe Técnico: Sistema de Archivos Distribuido Basado en Etiquetas (TBFS)

1. Arquitectura: El Problema de Cómo Diseñar el Sistema Organización del Sistema Distribuido

El diseño de un sistema de archivos distribuido basado en etiquetas presenta desafíos fundamentales que requieren una organización arquitectónica cuidadosamente planificada. La naturaleza distribuida del sistema impone la necesidad de separar responsabilidades para lograr escalabilidad, mantenibilidad y tolerancia a fallos.

La arquitectura del sistema TBFS se fundamenta en el principio de separación de concerns, organizándose en tres capas principales que abordan problemas distintos pero complementarios. Esta división responde a la necesidad de manejar de forma independiente el descubrimiento de servicios, la gestión de metadatos y el almacenamiento físico, cada uno con sus propios requisitos de consistencia, disponibilidad y rendimiento.

La primera capa, denominada **Capa de Descubrimiento**, se materializa mediante un servicio de registro distribuido que mantiene un catálogo dinámico de todos los servicios activos en el sistema. Esta capa resuelve el problema fundamental de cómo los componentes del sistema pueden localizarse mutuamente en un entorno distribuido donde las direcciones y disponibilidad de servicios cambian constantemente. La decisión de implementar esta capa como un servicio independiente surge de la necesidad de desacoplar el descubrimiento de servicios de la lógica de negocio, permitiendo que los servicios se registren y descubran dinámicamente sin requerir configuración estática.

La segunda capa, **Capa de Metadatos**, se encarga exclusivamente de la gestión de información descriptiva sobre los archivos: nombres, etiquetas, identificadores únicos y la ubicación de sus réplicas físicas. Esta separación responde a una observación crítica: los metadatos y los datos físicos tienen características de acceso, patrones de uso y requisitos de consistencia fundamentalmente diferentes. Los metadatos requieren consultas frecuentes y rápidas, actualizaciones atómicas y consistencia fuerte, mientras que los datos físicos requieren optimización para operaciones de lectura y escritura de grandes volúmenes. Al separar estas responsabilidades, el sistema puede optimizar cada componente de forma independiente.

La tercera capa, **Capa de Almacenamiento**, se compone de múltiples nodos especializados en el almacenamiento físico de archivos. Esta capa opera de forma autónoma pero coordinada, respondiendo a instrucciones de la capa de metadatos mientras gestiona localmente el almacenamiento en disco. La decisión de mantener múltiples nodos de almacenamiento independientes permite distribuir la carga, proporcionar redundancia y escalar horizontalmente la capacidad de almacenamiento sin afectar la gestión de metadatos.

Esta organización en capas no es arbitraria; responde directamente a los principios de diseño de sistemas distribuidos que buscan minimizar el acoplamiento entre componentes mientras maximizan la cohesión interna de cada capa. Cada capa puede evolucionar, escalar y fallar de forma independiente, proporcionando al sistema una robustez que sería difícil de lograr con una arquitectura monolítica.

Roles del Sistema

La asignación de roles específicos a cada componente del sistema deriva de la necesidad de establecer responsabilidades claras y bien definidas. En un sistema distribuido, la ambigüedad en las responsabilidades conduce inevitablemente a inconsistencias, condiciones de carrera y dificultades en el mantenimiento.

El **Registry Service** asume el rol de directorio centralizado del sistema, manteniendo un registro distribuido de todos los servicios activos. Este rol emerge de la necesidad de resolver el problema de descubrimiento de servicios en un entorno donde los servicios pueden iniciarse, detenerse o fallar en cualquier momento. El Registry no solo mantiene un catálogo estático, sino que implementa mecanismos de detección de servicios inactivos y eliminación automática, garantizando que el catálogo refleje siempre el estado actual del sistema. La decisión de implementar este servicio con múltiples réplicas y consenso distribuido responde a la necesidad de alta disponibilidad: si el Registry falla, todo el sistema perdería la capacidad de descubrir servicios.

El **MetaNameNode** desempeña el rol de coordinador central del sistema, gestionando exclusivamente metadatos y tomando decisiones sobre dónde y cómo almacenar los archivos. Este rol se justifica por la necesidad de mantener una visión global y consistente del sistema de archivos. El MetaNameNode no almacena archivos físicos, una decisión arquitectónica fundamental que permite que este componente sea ligero, rápido en consultas y fácilmente replicable. Al concentrar la lógica de decisión en este componente, el sistema puede implementar estrategias sofisticadas de asignación de réplicas, balanceo de carga y recuperación ante fallos, sin que cada nodo de almacenamiento necesite conocer el estado global del sistema.

Los **DataNodes** asumen el rol de almacenamiento especializado, enfocándose exclusivamente en operaciones de lectura y escritura de archivos físicos. Esta especialización permite optimizar estos nodos para operaciones de entrada/salida de disco, sin la sobrecarga de gestionar metadatos o tomar decisiones de coordinación. La decisión de mantener múltiples DataNodes independientes responde a varios requisitos: necesidad de capacidad de almacenamiento escalable, requisito de redundancia para tolerancia a fallos, y deseo de distribuir la carga de operaciones de entrada/salida entre múltiples dispositivos físicos.

El **Cliente** desempeña el rol de interfaz de usuario, proporcionando tanto una interfaz gráfica web como una interfaz de línea de comandos. Este rol es esencialmente pasivo en términos de coordinación del sistema, pero activo en términos

de iniciar operaciones. La decisión de mantener el cliente como un componente separado permite que múltiples usuarios accedan al sistema simultáneamente sin interferir entre sí, y permite que el cliente se ejecute en máquinas diferentes a las que ejecutan los servicios del sistema.

Distribución de Servicios en Red Docker Swarm

La decisión de utilizar Docker Swarm como plataforma de despliegue responde a la necesidad de gestionar un sistema distribuido complejo de forma coherente y escalable. Docker Swarm proporciona abstracciones que simplifican significativamente el despliegue y gestión de servicios distribuidos, mientras proporciona mecanismos nativos para redes distribuidas, descubrimiento de servicios y balanceo de carga.

La red overlay de Docker Swarm (`tbfs_net`) resuelve el problema de comunicación entre servicios que pueden ejecutarse en diferentes nodos físicos del cluster. Esta red virtual distribuida permite que los servicios se comuniquen usando nombres lógicos, independientemente de en qué nodo físico se ejecuten. Esta abstracción es crucial porque permite que el sistema se redistribuya automáticamente cuando un nodo falla o cuando se agregan nuevos nodos al cluster, sin requerir reconfiguración de los servicios.

La distribución de servicios en el swarm sigue un patrón de agrupación lógica: servicios relacionados se agrupan conceptualmente (por ejemplo, todos los nodos del Registry forman un cluster), pero se ejecutan como servicios independientes. Esta organización permite que Docker Swarm distribuya automáticamente los contenedores entre los nodos disponibles, proporcionando balanceo de carga y tolerancia a fallos a nivel de infraestructura.

Los volúmenes persistentes asociados a cada servicio resuelven el problema de persistencia de datos en un entorno donde los contenedores pueden ser recreados o movidos entre nodos. La decisión de usar volúmenes nombrados permite que los datos persistan independientemente del ciclo de vida de los contenedores, garantizando que la información crítica no se pierda cuando los servicios se reinician o se redistribuyen.

La exposición de puertos específicos en cada servicio responde a la necesidad de acceso externo para clientes y herramientas de administración, mientras que la comunicación interna entre servicios utiliza la red overlay privada. Esta separación de redes públicas y privadas proporciona una capa adicional de seguridad, limitando el acceso directo a servicios internos mientras permite acceso controlado a servicios que deben ser accesibles externamente.

2. Procesos: El Problema de Cuántos Programas o Servicios Posee el Sistema

Tipos de Procesos dentro del Sistema

La identificación y clasificación de los tipos de procesos en el sistema responde a la necesidad de comprender las diferentes responsabilidades computacionales y cómo estas se materializan en programas ejecutables. Cada tipo de proceso representa una unidad funcional distinta con requisitos específicos de recursos, patrones de comunicación y ciclos de vida.

Los **procesos de Registry** representan instancias del servicio de descubrimiento, cada una ejecutando un servidor web independiente que mantiene un registro en memoria de servicios activos. La decisión de mantener el registro en memoria, en lugar de persistirlo en disco, responde a la naturaleza efímera de la información: el estado de los servicios cambia frecuentemente y la pérdida de esta información al reiniciar es aceptable, ya que los servicios se re-registrarán automáticamente. Estos procesos implementan algoritmos de consenso distribuido para mantener consistencia entre múltiples instancias, requiriendo hilos de ejecución adicionales para manejar elecciones de líder, replicación de estado y limpieza periódica de servicios inactivos.

Los **procesos de MetaNameNode** encapsulan la lógica de gestión de metadatos, ejecutando servidores web que interactúan con bases de datos locales para almacenar y consultar información sobre archivos. La decisión de usar bases de datos locales en lugar de una base de datos centralizada responde a la necesidad de rendimiento en consultas frecuentes y a la simplificación de la arquitectura: cada réplica mantiene su propia copia de los metadatos, permitiendo que las consultas de lectura se distribuyan entre réplicas sin crear un cuello de botella centralizado. Estos procesos requieren hilos adicionales para monitorear el estado de los DataNodes, gestionar procesos de re-replicación automática y participar en elecciones de líder.

Los **procesos de DataNode** representan instancias especializadas en almacenamiento físico, ejecutando servidores web ligeros que responden a solicitudes de lectura y escritura de archivos. La simplicidad de estos procesos es intencional: al minimizar la lógica de negocio en estos nodos, se maximiza el rendimiento en operaciones de entrada/salida de disco, que es su función principal. Estos procesos implementan hilos para registro automático con el MetaNameNode y envío periódico de heartbeats, pero la mayor parte de su tiempo de ejecución se dedica a operaciones de entrada/salida bloqueantes.

Los **procesos de Cliente** difieren fundamentalmente de los procesos de servidor: mientras que los procesos de servidor están diseñados para ejecutarse indefinidamente y responder a solicitudes, los procesos de cliente tienen un ciclo de vida acotado y están orientados a la interacción con el usuario. El cliente web proporciona una interfaz gráfica interactiva que se ejecuta en un servidor web especializado, mientras que el cliente de línea de comandos ejecuta scripts

que realizan operaciones específicas y luego terminan. Esta diferencia refleja los diferentes casos de uso: el cliente web está diseñado para sesiones interactivas prolongadas, mientras que el cliente CLI está diseñado para automatización y scripting.

Organización o Agrupación de los Procesos

La organización de procesos en instancias separadas responde a principios fundamentales de diseño de sistemas distribuidos: aislamiento de fallos, escalabilidad independiente y separación de responsabilidades. Cada servicio se encapsula en su propio contenedor, proporcionando un límite claro de responsabilidad y un punto de fallo independiente.

La decisión de ejecutar múltiples réplicas del mismo tipo de servicio en instancias separadas, en lugar de en una sola instancia con múltiples hilos, responde a varias consideraciones. Primero, el aislamiento de fallos: si una réplica falla, las otras continúan operando independientemente. Segundo, la escalabilidad: cada instancia puede ejecutarse en un nodo físico diferente, permitiendo que el sistema aproveche múltiples CPUs y sistemas de almacenamiento. Tercero, la independencia de despliegue: cada instancia puede actualizarse, reiniciarse o reconfigurarse sin afectar a las demás.

La arquitectura de microservicios que emerge de esta organización proporciona beneficios adicionales. Cada servicio puede desarrollarse, probarse y desplegarse independientemente, permitiendo que diferentes equipos trabajen en paralelo sin interferencias. La escalabilidad selectiva permite agregar más instancias de servicios que experimentan alta carga (como DataNodes) sin necesidad de escalar servicios que no requieren más capacidad (como el Registry, que tiene requisitos de recursos relativamente bajos).

Esta organización también facilita la tolerancia a fallos parciales: si un servicio falla, los demás continúan operando. Por ejemplo, si un DataNode falla, el sistema puede continuar sirviendo archivos desde otros DataNodes, y si un MetaNameNode falla, las otras réplicas pueden continuar procesando solicitudes. Esta resiliencia sería difícil de lograr con una arquitectura monolítica donde todos los componentes comparten el mismo proceso.

Patrones de Diseño con Respeto al Desempeño

La elección de patrones de diseño para el manejo de concurrencia y operaciones asíncronas responde a las diferentes características de las operaciones que el sistema debe realizar. No existe un único patrón óptimo para todas las situaciones; en su lugar, el sistema utiliza una combinación de patrones, cada uno apropiado para su contexto específico.

El **modelo de hilos (threading)** se utiliza para operaciones que requieren ejecución continua en segundo plano pero que no bloquean las operaciones principales del servidor. Los hilos de monitoreo, por ejemplo, ejecutan bucles que

verifican periódicamente el estado de otros componentes, permitiendo que el servidor principal continúe respondiendo a solicitudes mientras se realizan estas verificaciones. Los hilos de heartbeat mantienen comunicación periódica con otros servicios, una operación que debe ocurrir continuamente pero que no debe bloquear el procesamiento de solicitudes de clientes. La decisión de usar hilos en lugar de procesos separados para estas tareas responde a la necesidad de compartir estado y recursos: los hilos dentro del mismo proceso pueden acceder a estructuras de datos compartidas (protegidas por locks) de forma más eficiente que los procesos separados.

El **modelo asíncrono** se aplica a operaciones de entrada/salida que pueden bloquearse esperando respuestas de red o disco. Las operaciones asíncronas permiten que el servidor inicie una operación de entrada/salida y luego continúe procesando otras solicitudes mientras espera la respuesta. Esta capacidad es crucial para el rendimiento: un servidor que procesa solicitudes de forma síncrona solo puede manejar una solicitud a la vez por hilo, mientras que un servidor asíncrono puede manejar miles de solicitudes concurrentes con un número relativamente pequeño de hilos. La decisión de usar operaciones asíncronas para peticiones HTTP y operaciones de archivos responde directamente a la necesidad de alto rendimiento y utilización eficiente de recursos.

Los **procesos independientes** representan el nivel más alto de aislamiento y paralelismo. Cada servicio se ejecuta como un proceso completamente independiente, permitiendo que el sistema operativo los distribuya entre diferentes núcleos de CPU y, en un entorno de Docker Swarm, entre diferentes nodos físicos. Esta organización proporciona el máximo aprovechamiento de recursos hardware disponibles y el mejor aislamiento de fallos. La decisión de usar procesos independientes para servicios diferentes responde a la necesidad de escalabilidad horizontal: agregar más nodos al cluster permite ejecutar más instancias de servicios, proporcionando capacidad adicional sin límites prácticos.

El **patrón líder-seguidor** implementado en el Registry y MetaNameNode resuelve el problema de coordinación en sistemas distribuidos donde múltiples réplicas deben mantener consistencia. Este patrón garantiza que solo una réplica procese operaciones de escritura en un momento dado, evitando conflictos y condiciones de carrera. Los seguidores replican las operaciones del líder, manteniendo sus propios estados sincronizados. La elección automática de un nuevo líder cuando el líder actual falla proporciona alta disponibilidad sin intervención manual. Este patrón es esencial para mantener la consistencia en un sistema distribuido donde las réplicas pueden tener diferentes estados debido a latencia de red o fallos temporales.

3. Comunicación: El Problema de Cómo Enviar Información Mediante la Red

Tipo de Comunicación

La elección del protocolo y estilo de comunicación en un sistema distribuido tiene implicaciones profundas en la simplicidad de implementación, la interoperabilidad, el rendimiento y la mantenibilidad del sistema. El sistema TBFS utiliza REST (Representational State Transfer) como protocolo principal de comunicación, una decisión que responde a múltiples consideraciones de diseño.

REST se fundamenta en el protocolo HTTP, un estándar universalmente soportado que proporciona una base sólida y bien entendida para la comunicación entre servicios. Esta elección elimina la necesidad de implementar protocolos personalizados o bibliotecas especializadas, reduciendo significativamente la complejidad del sistema. La naturaleza stateless de REST simplifica el diseño de servidores: cada petición contiene toda la información necesaria para procesarla, sin requerir que el servidor mantenga estado de sesión entre peticiones. Esta característica es particularmente valiosa en un sistema distribuido donde las peticiones pueden ser dirigidas a diferentes instancias de un servicio.

La serialización de datos en formato JSON proporciona un equilibrio entre legibilidad humana y eficiencia de procesamiento. JSON es auto-descriptivo, permitiendo que los desarrolladores comprendan fácilmente la estructura de los datos sin documentación adicional, mientras que su simplicidad permite parsing eficiente. Esta elección facilita la depuración y el desarrollo, ya que los mensajes pueden inspeccionarse directamente sin herramientas especializadas.

La validación de datos mediante modelos de datos estructurados proporciona una capa adicional de robustez, garantizando que los datos recibidos cumplan con las expectativas del sistema antes de procesarlos. Esta validación previene errores que podrían propagarse a través del sistema y causar comportamientos inesperados o fallos.

La documentación automática de APIs que surge del uso de frameworks modernos que soportan REST proporciona beneficios adicionales: los desarrolladores pueden comprender y probar las APIs sin necesidad de documentación externa, y los cambios en las APIs se reflejan automáticamente en la documentación, reduciendo la posibilidad de desincronización entre implementación y documentación.

Comunicación Cliente-Servidor

La comunicación entre clientes y servidores en el sistema TBFS sigue un patrón de solicitud-respuesta, donde los clientes inician todas las interacciones y los servidores responden a estas solicitudes. Esta asimetría es intencional y responde a la naturaleza de las operaciones: los clientes representan usuarios o procesos automatizados que necesitan realizar operaciones específicas, mientras

que los servidores están diseñados para responder a estas solicitudes de forma eficiente.

La comunicación entre clientes y el Registry Service se limita a operaciones de lectura: los clientes consultan el Registry para descubrir servicios disponibles, pero no modifican el estado del Registry directamente. Esta restricción responde a la necesidad de mantener la integridad del catálogo de servicios: solo los servicios mismos pueden registrarse, evitando que clientes maliciosos o erróneos modifiquen el catálogo. El failover automático entre múltiples nodos del Registry proporciona alta disponibilidad: si un nodo del Registry no responde, el cliente puede intentar con otro nodo automáticamente.

La comunicación entre clientes y el MetaNameNode abarca un rango más amplio de operaciones: lectura (listar archivos, descargar archivos), escritura (subir archivos, agregar etiquetas) y eliminación (eliminar archivos, eliminar etiquetas). La redirección automática hacia el líder cuando un cliente se conecta a un seguidor resuelve el problema de descubrimiento del líder sin requerir que los clientes implementen lógica compleja de elección de líder. Esta redirección es transparente para el cliente, proporcionando una experiencia de usuario fluida mientras permite que el sistema distribuya la carga de lectura entre múltiples réplicas.

La ausencia de comunicación directa entre clientes y DataNodes es una decisión arquitectónica fundamental. Esta restricción centraliza la lógica de coordinación en el MetaNameNode, simplificando significativamente el diseño del sistema. Los clientes no necesitan conocer la ubicación de los archivos físicos, cómo están replicados, o qué DataNodes están disponibles; toda esta complejidad se abstrae en el MetaNameNode. Esta centralización también proporciona beneficios de seguridad: los DataNodes no necesitan exponer interfaces públicas, reduciendo la superficie de ataque del sistema.

Comunicación Servidor-Servidor

La comunicación entre servidores en el sistema TBFS es más compleja que la comunicación cliente-servidor, ya que involucra coordinación, replicación y sincronización de estado. Estas operaciones requieren garantías de consistencia y confiabilidad que van más allá de las operaciones cliente-servidor típicas.

La comunicación entre nodos del Registry involucra operaciones de consenso distribuido: elección de líder mediante votación y replicación de estado del catálogo de servicios. Estas operaciones son críticas para el funcionamiento del sistema y requieren garantías de que las actualizaciones se propaguen correctamente a todas las réplicas. La implementación de endpoints internos específicos para estas operaciones permite que el sistema optimice estas comunicaciones sin exponer detalles de implementación a clientes externos.

La comunicación entre réplicas del MetaNameNode sigue un patrón similar: replicación de operaciones de metadatos desde el líder hacia los seguidores, y

heartbeats periódicos para mantener la coherencia del estado del cluster. Estas comunicaciones deben ser confiables y eficientes, ya que ocurren frecuentemente y son esenciales para mantener la consistencia del sistema. La decisión de usar endpoints internos separados permite que el sistema implemente optimizaciones específicas para estas operaciones, como timeouts más cortos o reintentos automáticos.

La comunicación del MetaNameNode hacia los DataNodes abarca una gama de operaciones: registro de nuevos DataNodes, recepción de heartbeats periódicos, envío de archivos para almacenamiento, solicitud de lectura de archivos y eliminación de archivos. Estas operaciones representan el flujo principal de datos en el sistema y deben ser eficientes y confiables. La decisión de que el MetaNameNode inicie todas estas comunicaciones (en lugar de permitir que los DataNodes inicien comunicaciones) simplifica el diseño y proporciona un punto centralizado de control.

La comunicación de los DataNodes hacia el Registry se limita a consultas para descubrir el MetaNameNode líder. Esta limitación responde a la necesidad de mantener la arquitectura simple: los DataNodes no necesitan registrarse directamente en el Registry, ya que se registran en el MetaNameNode, que a su vez se registra en el Registry. Esta jerarquía de registro simplifica la gestión del catálogo de servicios y mantiene responsabilidades claras.

Comunicación entre Procesos

La comunicación entre procesos dentro del mismo servicio (intra-proceso) y entre servicios diferentes (inter-proceso) presenta desafíos y oportunidades diferentes, cada una requiriendo mecanismos apropiados.

La comunicación intra-proceso utiliza mecanismos de memoria compartida protegidos por primitivas de sincronización. Esta comunicación es extremadamente eficiente pero requiere cuidado para evitar condiciones de carrera y deadlocks. La decisión de usar locks (mutex) para proteger estructuras de datos compartidas responde a la necesidad de garantizar que las actualizaciones concurrentes no corrompan el estado. El uso de threading para operaciones concurrentes dentro del mismo proceso permite aprovechar múltiples núcleos de CPU mientras mantiene la eficiencia de la comunicación en memoria.

La comunicación inter-proceso utiliza exclusivamente peticiones HTTP REST, incluso cuando los procesos se ejecutan en el mismo nodo físico. Esta decisión puede parecer ineficiente, pero proporciona beneficios significativos: desacoplamiento completo entre servicios, independencia de implementación (cada servicio puede implementarse en diferentes lenguajes o frameworks), y facilidad de escalabilidad (los servicios pueden moverse entre nodos sin cambios en la comunicación). Esta uniformidad en el mecanismo de comunicación simplifica significativamente el diseño del sistema y facilita el mantenimiento y la evolución futura.

4. Coordinación: El Problema de Poner Todos los Servicios de Acuerdo

Sincronización de Acciones

La coordinación efectiva en un sistema distribuido requiere mecanismos que garanticen que las acciones de múltiples componentes ocurran en el orden correcto y con las garantías de consistencia apropiadas. La sincronización de acciones en el sistema TBFS responde a diferentes necesidades según el tipo de operación y los requisitos de consistencia asociados.

La sincronización de operaciones de escritura se fundamenta en el principio de que solo un componente debe tener autoridad para modificar el estado del sistema en un momento dado. Esta restricción previene condiciones de carrera donde múltiples componentes intentan modificar el mismo recurso simultáneamente, lo que podría resultar en estados inconsistentes o pérdida de datos. El sistema implementa esta sincronización mediante el patrón líder-seguidor: todas las operaciones de escritura deben ser procesadas por el MetaNameNode líder, mientras que los nodos seguidores redirigen automáticamente estas solicitudes al líder. Esta centralización de autoridad de escritura simplifica significativamente el problema de mantener consistencia, ya que elimina la necesidad de resolver conflictos entre múltiples escritores concurrentes.

La sincronización de replicación aborda el problema de cómo propagar cambios desde el líder hacia los seguidores de forma que se mantenga la consistencia entre réplicas. El sistema implementa un modelo donde el líder replica cada operación a todos los seguidores antes de confirmar la operación al cliente. Esta estrategia requiere que una mayoría de réplicas (quorum) confirme la recepción de la operación antes de que se considere completada. Este requisito de quorum es fundamental: garantiza que incluso si algunas réplicas fallan, el sistema puede continuar operando mientras una mayoría permanezca activa. La decisión de requerir confirmación de mayoría antes de confirmar al cliente proporciona consistencia fuerte: el cliente puede estar seguro de que su operación se ha propagado a suficientes réplicas para sobrevivir a fallos posteriores.

La sincronización de heartbeats proporciona un mecanismo para mantener la coherencia del estado del sistema sin requerir comunicación constante y exhaustiva. Los DataNodes envían señales periódicas de vida al MetaNameNode, indicando no solo que están activos, sino también información sobre su estado actual, como espacio disponible. Esta comunicación periódica permite que el MetaNameNode mantenga una visión actualizada del estado del sistema sin requerir consultas constantes que consumirían recursos significativos. El intervalo de heartbeats representa un equilibrio entre la frescura de la información y el consumo de recursos: intervalos muy cortos proporcionan información más actualizada pero consumen más ancho de banda y recursos de procesamiento, mientras que intervalos muy largos pueden resultar en detección tardía de fallos.

La sincronización de elección de líder resuelve el problema de cómo seleccionar

un nuevo líder cuando el líder actual falla o se vuelve inaccesible. El sistema implementa un mecanismo que previene elecciones frecuentes mediante un período de cooldown entre elecciones. Esta prevención es crucial porque las elecciones de líder son operaciones costosas que requieren comunicación entre múltiples nodos y pueden interrumpir el procesamiento normal de operaciones. El uso de términos incrementales garantiza que las elecciones ocurran en un orden bien definido, previniendo situaciones donde múltiples nodos creen que son líderes simultáneamente. El requisito de mayoría en las votaciones garantiza que solo un candidato con suficiente apoyo pueda convertirse en líder, previniendo divisiones del cluster donde múltiples líderes procesan operaciones de forma independiente.

Acceso Exclusivo a Recursos

En un sistema donde múltiples hilos o procesos pueden acceder simultáneamente a recursos compartidos, es esencial garantizar que las operaciones que modifican el estado ocurran de forma atómica y sin interferencia. El problema de acceso exclusivo a recursos se manifiesta en diferentes niveles del sistema, cada uno requiriendo mecanismos apropiados de protección.

La protección de bases de datos responde a la necesidad de garantizar que las operaciones de lectura y escritura en bases de datos no interfieran entre sí de formas que corrompan los datos. El sistema utiliza mecanismos de bloqueo (locks) que garantizan que solo un hilo pueda modificar la base de datos en un momento dado, mientras que múltiples hilos pueden leer simultáneamente. Esta distinción entre locks de lectura y escritura optimiza el rendimiento: las operaciones de lectura, que son más frecuentes, no se bloquean entre sí, mientras que las operaciones de escritura obtienen acceso exclusivo para garantizar la integridad. La implementación de transacciones atómicas para operaciones complejas garantiza que múltiples cambios relacionados ocurran como una unidad indivisible: o todos los cambios se aplican, o ninguno se aplica, previniendo estados intermedios inconsistentes.

La protección del estado del cluster aborda el problema de cómo mantener información sobre el estado del sistema distribuido (como qué nodo es el líder, qué término está activo, qué nodos son peers) de forma que múltiples hilos puedan acceder y modificar esta información sin crear condiciones de carrera. El sistema utiliza locks específicos para proteger estas estructuras de estado, garantizando que las actualizaciones sean atómicas. La implementación de timeouts en la adquisición de locks previene deadlocks: si un hilo no puede adquirir un lock dentro de un tiempo razonable, puede abortar la operación o intentar una estrategia alternativa, en lugar de bloquearse indefinidamente esperando un lock que puede no liberarse nunca.

La gestión de locks en el sistema sigue principios que buscan minimizar el tiempo durante el cual los recursos están bloqueados, maximizando así el paralelismo y el rendimiento. Los locks se adquieren solo cuando es absolutamente necesario y se liberan tan pronto como sea posible. Las operaciones que no requieren

modificar estado compartido se diseñan para no requerir locks, permitiendo que ocurran completamente en paralelo. Esta filosofía de diseño minimiza la contención de recursos y permite que el sistema maneje altas cargas de trabajo concurrentes de forma eficiente.

Toma de Decisiones Distribuidas

En un sistema distribuido, muchas decisiones requieren el consenso de múltiples componentes. La toma de decisiones distribuidas presenta desafíos únicos: los componentes pueden tener información diferente debido a latencia de red, pueden fallar durante el proceso de decisión, o pueden estar temporalmente desconectados. El sistema TBFS implementa varios mecanismos para abordar estos desafíos.

La elección de líder mediante algoritmos de consenso distribuido resuelve el problema fundamental de cómo seleccionar un coordinador cuando múltiples candidatos pueden competir por el rol. El algoritmo implementado permite que cualquier nodo que detecte la ausencia de un líder inicie una elección, pero garantiza que solo un candidato con suficiente apoyo pueda convertirse en líder. La votación se basa en términos incrementales, donde cada elección tiene un término mayor que la anterior, garantizando que las elecciones ocurran en un orden bien definido. El candidato que recibe una mayoría de votos se convierte en líder y mantiene su posición mediante el envío periódico de heartbeats a los seguidores. Si el líder falla o se vuelve inaccesible, los seguidores detectan la ausencia de heartbeats e inician una nueva elección. Este mecanismo proporciona alta disponibilidad: el sistema puede continuar operando incluso cuando el líder falla, seleccionando automáticamente un nuevo líder sin intervención manual.

El consenso en replicación aborda el problema de cómo garantizar que las operaciones se propaguen correctamente a todas las réplicas antes de considerarse completadas. El líder propone una operación y espera confirmación de una mayoría de seguidores antes de confirmar la operación al cliente. Este requisito de mayoría es fundamental: garantiza que incluso si algunas réplicas fallan durante o después de la replicación, el sistema puede continuar operando con las réplicas restantes, y las réplicas que se recuperen pueden sincronizarse con el estado actual. Si una réplica falla durante una operación, el sistema puede continuar con la mayoría restante, pero la operación solo se confirma si la mayoría puede confirmar. Esta estrategia proporciona un equilibrio entre disponibilidad y consistencia: el sistema puede continuar operando con fallos parciales, pero solo confirma operaciones cuando puede garantizar que se han propagado a suficientes réplicas.

La decisión de asignación de réplicas representa un problema de optimización distribuida: el MetaNameNode debe decidir en qué DataNodes almacenar cada archivo, considerando múltiples factores como espacio disponible, carga actual, estado de salud de los DataNodes, y distribución de carga. El sistema imple-

menta un algoritmo que evalúa todos los DataNodes disponibles, excluye aquellos que están en proceso de drenaje o marcados como inactivos, y selecciona DataNodes que maximicen la distribución de carga y minimicen el riesgo de pérdida de datos. La decisión utiliza información del hash del archivo para proporcionar una distribución determinística: archivos con hashes similares tienden a distribuirse de forma similar, pero la selección final considera el estado actual del sistema. Esta combinación de determinismo y adaptabilidad permite que el sistema mantenga una distribución balanceada mientras responde dinámicamente a cambios en el estado del sistema.

5. Nombrado y Localización: El Problema de Dónde se Encuentra un Recurso y Cómo Llegar al Mismo

Identificación de Datos y Servicios

La identificación única y confiable de recursos en un sistema distribuido es un problema fundamental que afecta todas las operaciones del sistema. Sin mecanismos robustos de identificación, el sistema no puede localizar, acceder o gestionar recursos de forma confiable. El sistema TBFS implementa múltiples niveles de identificación, cada uno apropiado para diferentes contextos y necesidades.

La identificación de archivos utiliza una estrategia de múltiples identificadores complementarios. El nombre original proporcionado por el usuario sirve como identificador primario desde la perspectiva del usuario, pero este identificador no es suficiente para el sistema interno debido a la posibilidad de nombres duplicados o ambiguos. El sistema genera un hash criptográfico del contenido del archivo, proporcionando un identificador único que es inherente al contenido: dos archivos con contenido idéntico tendrán el mismo hash, independientemente de sus nombres. Esta propiedad del hash permite deduplicación automática: si un usuario intenta subir un archivo que ya existe (identificado por su hash), el sistema puede reconocer esto y evitar almacenar duplicados. El hash también proporciona verificación de integridad: cuando se lee un archivo, el sistema puede verificar que el contenido no ha sido corrompido comparando el hash calculado del contenido leído con el hash almacenado en los metadatos. Adicionalmente, el sistema asigna un identificador numérico único en la base de datos de metadatos, proporcionando una referencia eficiente para operaciones internas y relaciones con otros datos.

La identificación de servicios responde a la necesidad de localizar y comunicarse con componentes del sistema en un entorno donde las direcciones físicas pueden cambiar. El sistema utiliza identificadores lógicos (NODE_ID) que son estables a través de reinicios y redistribuciones, independientemente de dónde se ejecute físicamente el servicio. Estos identificadores se mapean a nombres de servicio en Docker Swarm, que a su vez se resuelven a direcciones de red mediante el sistema de resolución de nombres del swarm. Esta capa de abstracción permite que los servicios se muevan entre nodos físicos sin requerir reconfiguración: el identificador lógico permanece constante, pero la resolución de nombres se ac-

tualiza automáticamente cuando el servicio se mueve. La URL completa que combina protocolo, host y puerto proporciona una referencia completa para comunicación, pero esta URL se construye dinámicamente a partir de los identificadores lógicos, no se almacena estáticamente.

La identificación de DataNodes utiliza identificadores únicos que persisten a través de reinicios, permitiendo que el MetaNameNode rastree qué archivos están almacenados en cada DataNode incluso cuando los DataNodes se reinician o se mueven entre nodos físicos. Esta persistencia es crucial para la gestión de réplicas: el sistema debe poder identificar qué DataNode contiene qué archivos para poder coordinar operaciones de lectura, escritura y re-replicación. La estabilidad de estos identificadores permite que el sistema mantenga registros de asignación de réplicas que permanecen válidos incluso cuando los DataNodes experimentan interrupciones temporales.

Ubicación de Datos y Servicios

La ubicación física de datos y servicios en un sistema distribuido puede cambiar debido a redistribución de carga, fallos de nodos, o decisiones de optimización. El sistema TBFS implementa estrategias que permiten que los datos y servicios se muevan mientras mantienen la capacidad del sistema de localizarlos y acceder a ellos.

La ubicación de metadatos sigue una estrategia de replicación completa: cada réplica del MetaNameNode mantiene una copia completa de todos los metadatos en una base de datos local. Esta estrategia responde a varios requisitos: primero, permite que cualquier réplica responda a consultas de lectura sin requerir comunicación con otras réplicas, proporcionando alto rendimiento y disponibilidad. Segundo, proporciona redundancia: si una réplica falla, las otras contienen toda la información necesaria para continuar operando. Tercero, simplifica la arquitectura: no se requiere una base de datos centralizada que podría convertirse en un cuello de botella o punto único de fallo. La ubicación de estas bases de datos en volúmenes persistentes asociados a cada réplica garantiza que los metadatos persistan incluso cuando los contenedores se recrean, proporcionando durabilidad a través de reinicios y redistribuciones.

La ubicación de archivos físicos se distribuye entre múltiples DataNodes, con cada archivo almacenado en exactamente tres DataNodes diferentes. La ubicación específica de cada archivo se determina mediante un algoritmo que considera el estado actual del sistema, pero la estructura de almacenamiento dentro de cada DataNode organiza archivos por los primeros caracteres de su hash. Esta organización facilita operaciones de búsqueda y gestión dentro de cada DataNode, mientras que la distribución entre DataNodes proporciona redundancia y distribuye la carga. La decisión de no almacenar archivos en el MetaNameNode responde a la necesidad de separar responsabilidades: el MetaNameNode se optimiza para consultas rápidas de metadatos, mientras que los DataNodes se optimizan para operaciones de entrada/salida de archivos grandes.

La ubicación de servicios en Docker Swarm se gestiona mediante el sistema de orquestación del swarm, que distribuye contenedores entre nodos disponibles según políticas de colocación y disponibilidad de recursos. El Registry Service mantiene un catálogo de servicios activos que incluye información sobre cómo localizar cada servicio, pero la resolución real de nombres a direcciones físicas la realiza el sistema de red overlay de Docker Swarm. Esta separación de responsabilidades permite que el Registry mantenga información lógica sobre servicios mientras que Docker Swarm maneja los detalles de ubicación física y resolución de nombres. Los puertos expuestos en cada servicio proporcionan puntos de acceso desde fuera del swarm, pero la comunicación interna utiliza la red overlay que maneja automáticamente el enrutamiento entre servicios independientemente de su ubicación física.

Localización de Datos y Servicios

La localización de recursos en un sistema distribuido requiere mecanismos que permitan encontrar recursos específicos a partir de información parcial o lógica. El sistema TBFS implementa varios mecanismos de localización, cada uno apropiado para diferentes tipos de recursos y casos de uso.

El descubrimiento de servicios resuelve el problema de cómo los clientes y otros servicios pueden encontrar servicios disponibles en el sistema. Los clientes consultan el Registry Service para obtener listas de servicios activos, recibiendo información que incluye identificadores, URLs y estado de cada servicio. El Registry mantiene este catálogo actualizado mediante registro automático de servicios al iniciar y detección automática de servicios inactivos. El failover automático entre múltiples nodos del Registry proporciona alta disponibilidad: si un nodo del Registry no responde, los clientes pueden consultar otros nodos automáticamente. Esta capacidad de failover es crucial porque el Registry es un componente fundamental: sin acceso al Registry, los clientes no pueden descubrir otros servicios del sistema.

La localización del líder del MetaNameNode resuelve el problema de cómo los clientes pueden encontrar el nodo que tiene autoridad para procesar operaciones de escritura. El sistema implementa un mecanismo donde cualquier réplica del MetaNameNode puede responder a consultas sobre el estado del cluster, incluyendo información sobre qué nodo es el líder actual. Cuando un cliente se conecta a un seguidor para una operación de escritura, el seguidor redirige automáticamente al cliente hacia el líder. Esta redirección es transparente para el cliente y proporciona una experiencia fluida mientras permite que el sistema distribuya la carga de lectura entre múltiples réplicas. La información del líder se actualiza dinámicamente cuando ocurren elecciones de líder, garantizando que los clientes siempre sean dirigidos al líder actual, incluso cuando el líder cambia debido a fallos o redistribuciones.

La localización de réplicas de archivos aborda el problema de cómo encontrar las copias físicas de un archivo específico cuando se necesita leerlo o modificarlo.

El MetaNameNode mantiene un registro que mapea cada archivo (identificado por su ID) a los DataNodes donde están almacenadas sus réplicas, incluyendo información sobre el tipo de réplica (primaria, secundaria, terciaria). Cuando se solicita un archivo, el MetaNameNode consulta este registro y retorna una lista de DataNodes que contienen el archivo, ordenada por prioridad. El sistema intenta leer desde la réplica primaria primero, pero si esta falla o no está disponible, automáticamente intenta con réplicas secundarias. Este mecanismo de fallback proporciona alta disponibilidad: incluso si algunos DataNodes fallan, el sistema puede continuar sirviendo archivos desde réplicas en DataNodes activos.

La búsqueda por etiquetas representa un mecanismo de localización de nivel más alto, donde los usuarios identifican archivos mediante características descriptivas (etiquetas) en lugar de nombres o identificadores técnicos. El sistema implementa esta búsqueda mediante asociaciones almacenadas en la base de datos de metadatos, donde cada archivo puede tener múltiples etiquetas y cada etiqueta puede estar asociada a múltiples archivos. Las consultas filtran archivos que contienen todas las etiquetas especificadas, proporcionando una capacidad de búsqueda semántica que es más intuitiva para los usuarios que la búsqueda por nombres o identificadores técnicos. El MetaNameNode procesa estas consultas localmente en su base de datos, aprovechando índices para proporcionar respuestas rápidas incluso cuando hay grandes cantidades de archivos y etiquetas. Esta capacidad de búsqueda semántica es una característica distintiva del sistema que responde directamente al problema que el sistema está diseñado para resolver: la gestión de archivos mediante etiquetas en lugar de jerarquías de directorios tradicionales.

6. Consistencia y Replicación: El Problema de Solucionar los Problemas que Surgen a Partir de Tener Varias Copias de un Mismo Dato en el Sistema

Distribución de los Datos

La distribución de datos en un sistema de archivos distribuido presenta desafíos fundamentales que requieren estrategias cuidadosamente diseñadas. El sistema TBFS implementa diferentes estrategias de distribución según el tipo de dato y sus requisitos de acceso, consistencia y disponibilidad.

La distribución de metadatos sigue una estrategia de replicación completa, donde cada réplica del MetaNameNode mantiene una copia completa de todos los metadatos del sistema. Esta estrategia responde a varios requisitos críticos. Primero, las consultas de metadatos son extremadamente frecuentes: cada operación de lectura, búsqueda o listado requiere acceso a metadatos. Si los metadatos estuvieran centralizados en un solo nodo, este nodo se convertiría en un cuello de botella que limitaría severamente el rendimiento del sistema. Al replicar completamente los metadatos, el sistema puede distribuir las consultas entre múltiples réplicas, proporcionando escalabilidad horizontal

para operaciones de lectura. Segundo, la replicación completa proporciona alta disponibilidad: si una réplica falla, las otras pueden continuar procesando todas las consultas sin degradación funcional. Tercero, esta estrategia simplifica la arquitectura: no se requiere un sistema de particionamiento complejo que divida metadatos entre múltiples nodos, evitando problemas de consistencia que surgen cuando los datos relacionados están en diferentes particiones.

La distribución de archivos físicos utiliza una estrategia diferente: cada archivo se almacena en exactamente tres DataNodes diferentes, pero no todos los archivos se almacenan en los mismos DataNodes. Esta estrategia responde a la necesidad de balancear múltiples objetivos: proporcionar redundancia para tolerancia a fallos, distribuir la carga de almacenamiento entre múltiples nodos, y minimizar el riesgo de pérdida de datos. La decisión de usar exactamente tres réplicas representa un equilibrio entre redundancia y eficiencia: tres réplicas proporcionan tolerancia al fallo de un DataNode mientras mantienen un uso razonable de espacio de almacenamiento. Si se usaran más réplicas, se proporcionaría mayor redundancia pero a un costo significativamente mayor de almacenamiento. Si se usaran menos réplicas, se reduciría el uso de almacenamiento pero se aumentaría el riesgo de pérdida de datos.

La estrategia de distribución considera múltiples factores al decidir dónde almacenar cada archivo. El sistema evalúa el espacio disponible en cada DataNode, priorizando aquellos con más espacio libre para mantener un balance de carga. El estado de salud de los DataNodes es crucial: los DataNodes que están inactivos o en proceso de drenaje se excluyen de nuevas asignaciones. La distribución de carga se considera para evitar que algunos DataNodes se sobrecarguen mientras otros permanecen subutilizados. El algoritmo utiliza el hash del archivo para proporcionar una distribución determinística: archivos con contenido similar (y por tanto hashes similares) tienden a distribuirse de forma predecible, pero la selección final considera el estado dinámico del sistema para adaptarse a cambios en disponibilidad y carga.

Replicación

La replicación en el sistema TBFS no es simplemente una copia mecánica de datos; es un mecanismo fundamental que proporciona redundancia, alta disponibilidad y capacidad de recuperación ante fallos. Sin embargo, la replicación introduce complejidad: el sistema debe garantizar que las réplicas permanezcan consistentes, o al menos suficientemente consistentes para los requisitos del sistema.

Las réplicas de metadatos se mantienen mediante un proceso de replicación síncrona donde cada operación de escritura se propaga a todas las réplicas antes de confirmarse al cliente. Este modelo de replicación síncrona proporciona consistencia fuerte: todas las réplicas tienen exactamente el mismo estado después de cada operación completada. Esta consistencia fuerte es esencial para los metadatos porque las inconsistencias podrían resultar en pérdida de informa-

ción sobre archivos, etiquetas incorrectas, o ubicaciones incorrectas de réplicas. El requisito de que todas las réplicas confirmen antes de completar la operación garantiza que ninguna operación se considere completada hasta que esté segura en múltiples ubicaciones. Si una réplica falla durante una operación, la operación se aborta y se reintenta, garantizando que todas las réplicas activas mantengan el mismo estado.

Las réplicas de archivos físicos siguen un modelo diferente: el sistema requiere que al menos dos de las tres réplicas se almacenen exitosamente antes de confirmar la operación. Este modelo proporciona un equilibrio entre disponibilidad y consistencia: el sistema puede continuar operando incluso si una réplica falla durante la escritura, pero garantiza que al menos dos réplicas estén disponibles para lectura. Esta estrategia reconoce que las operaciones de archivos grandes pueden ser costosas y que fallos temporales de red o disco no deberían impedir que el sistema continúe operando. Si una réplica falla durante la escritura, el sistema puede continuar con las réplicas restantes y re-replicar a un nuevo DataNode cuando sea posible. Este modelo proporciona consistencia eventual: todas las réplicas eventualmente contendrán el mismo contenido, pero pueden estar temporalmente desincronizadas durante fallos o recuperaciones.

Las réplicas del Registry Service se mantienen mediante consenso distribuido, donde el líder propaga actualizaciones a los seguidores y requiere confirmación de mayoría antes de considerar completada una actualización. Este modelo garantiza que el catálogo de servicios permanezca consistente entre todas las réplicas del Registry, proporcionando una visión unificada del estado del sistema a todos los componentes que consultan el Registry. La consistencia del Registry es crucial porque otros componentes dependen de él para descubrir servicios: inconsistencias en el Registry podrían resultar en que diferentes componentes tengan visiones diferentes de qué servicios están disponibles, llevando a comportamientos erróneos o fallos.

Confiabilidad de las Réplicas Tras Actualización

La confiabilidad de las réplicas después de una actualización es un aspecto crítico de la replicación que determina qué tan bien el sistema puede garantizar la integridad y disponibilidad de los datos. El sistema TBFS implementa diferentes modelos de consistencia según el tipo de dato y sus requisitos.

El modelo de consistencia para metadatos implementa consistencia fuerte, donde todas las réplicas del MetaNameNode deben confirmar una operación antes de que se considere completada. Este modelo proporciona la garantía más fuerte: después de que una operación se completa, todas las réplicas activas tienen exactamente el mismo estado. Esta consistencia fuerte es esencial para metadatos porque las inconsistencias podrían resultar en pérdida de información crítica o comportamientos erróneos del sistema. Si una réplica falla durante una operación, la operación se aborta y se reintenta, garantizando que no se cree un estado donde algunas réplicas tienen la actualización y otras no. Los clientes

siempre leen de réplicas que han confirmado todas las operaciones previas, garantizando que siempre ven un estado consistente y actualizado.

El modelo de replicación de archivos físicos implementa consistencia eventual con garantías de disponibilidad. El sistema requiere que al menos dos de tres réplicas se almacenen exitosamente antes de confirmar una operación de escritura. Si una réplica falla durante la escritura, el sistema puede continuar con las réplicas restantes, proporcionando disponibilidad incluso durante fallos parciales. Las réplicas fallidas se re-repican automáticamente cuando se detecta el fallo, restaurando eventualmente el nivel completo de replicación. Este modelo reconoce que las operaciones de archivos grandes pueden ser costosas y que fallos temporales no deberían impedir que el sistema continúe operando. La consistencia eventual es aceptable para archivos físicos porque el contenido de un archivo no cambia después de su creación inicial: una vez que un archivo se almacena, su contenido permanece constante, por lo que las réplicas eventualmente convergerán al mismo contenido incluso si están temporalmente desincronizadas.

La verificación de integridad mediante hashes criptográficos proporciona una capa adicional de confiabilidad. Cada archivo se identifica mediante un hash SHA-256 de su contenido, proporcionando una verificación inherente de integridad. Cuando se lee un archivo, el sistema puede calcular el hash del contenido leído y compararlo con el hash almacenado en los metadatos. Si los hashes no coinciden, el sistema detecta corrupción de datos y puede intentar leer desde otra réplica. Esta verificación de integridad es crucial porque los sistemas de almacenamiento pueden experimentar corrupción de datos debido a fallos de hardware, errores de software, o problemas de red. La capacidad de detectar y recuperarse de corrupción de datos es esencial para mantener la confiabilidad del sistema a largo plazo.

La re-replicación automática proporciona un mecanismo para restaurar el nivel de replicación cuando se detectan fallos. El sistema monitorea continuamente el estado de los DataNodes, detectando cuando un DataNode deja de enviar heartbeats o se vuelve inaccesible. Cuando se detecta un DataNode inactivo, el sistema identifica todos los archivos que estaban almacenados en ese DataNode y los re-replica desde réplicas activas a nuevos DataNodes disponibles. Este proceso ocurre automáticamente en segundo plano, sin requerir intervención manual, y restaura el nivel de replicación deseado sin interrumpir las operaciones normales del sistema. La re-replicación automática es esencial para mantener la confiabilidad a largo plazo: sin este mecanismo, los fallos de DataNodes reducirían gradualmente el nivel de replicación hasta que el sistema ya no pudiera tolerar fallos adicionales.

7. Tolerancia a Fallos: El Problema de Para Qué Pasar Tanto Trabajo Distribuyendo Datos y Servicios si al Fallar una Componente del Sistema Todo se Viene Abajo

Respuesta a Errores

La tolerancia a fallos en un sistema distribuido no es simplemente una característica deseable; es un requisito fundamental que determina si el sistema puede proporcionar servicio confiable en entornos reales donde los componentes fallan regularmente. El sistema TBFS implementa múltiples mecanismos de detección y respuesta a fallos, cada uno apropiado para diferentes tipos de componentes y escenarios de fallo.

La detección de fallos utiliza múltiples mecanismos complementarios que proporcionan redundancia en la detección. Los heartbeats periódicos proporcionan un mecanismo proactivo donde los componentes envían señales regulares de vida, permitiendo que otros componentes detecten fallos por ausencia de estas señales. Este mecanismo es eficiente porque requiere comunicación mínima durante operación normal, pero proporciona detección rápida de fallos cuando un componente deja de responder. Los timeouts en peticiones HTTP proporcionan un mecanismo reactivo donde los componentes detectan fallos cuando las peticiones no reciben respuesta dentro de un tiempo razonable. Los endpoints de verificación de salud permiten que los componentes verifiquen explícitamente el estado de otros componentes cuando sea necesario. La combinación de estos mecanismos proporciona detección robusta: si un mecanismo falla o es demasiado lento, otros mecanismos pueden detectar el fallo.

El manejo de fallos del MetaNameNode responde al problema crítico de mantener la disponibilidad del coordinador central del sistema. Cuando el líder del MetaNameNode falla, los seguidores detectan la ausencia de heartbeats y automáticamente inician una elección de líder. Este proceso de recuperación es completamente automático y no requiere intervención manual, proporcionando alta disponibilidad incluso durante fallos del componente más crítico del sistema. El nuevo líder asume el control y continúa procesando operaciones, mientras que los clientes se redirigen automáticamente al nuevo líder. El tiempo de recuperación está diseñado para ser mínimo: el sistema puede detectar un fallo del líder y seleccionar un nuevo líder en un tiempo relativamente corto, minimizando la interrupción del servicio. Durante la transición, las operaciones de lectura pueden continuar desde réplicas seguidoras, mientras que las operaciones de escritura pueden experimentar una breve pausa hasta que se seleccione el nuevo líder.

El manejo de fallos de DataNodes aborda el problema de mantener disponibilidad de datos cuando los nodos de almacenamiento fallan. Cuando el MetaNameNode detecta que un DataNode está inactivo, identifica automáticamente todos los archivos que estaban almacenados en ese DataNode. El sistema entonces inicia un proceso de re-replicación automática que lee cada archivo afectado desde réplicas activas y los re-replica a nuevos DataNodes disponibles.

Este proceso ocurre en segundo plano y no bloquea las operaciones normales del sistema: los clientes pueden continuar leyendo archivos desde réplicas en DataNodes activos mientras la re-replicación ocurre. La capacidad de continuar sirviendo datos durante la re-replicación es crucial: sin esta capacidad, cada fallo de DataNode resultaría en pérdida temporal de acceso a los archivos afectados, lo cual sería inaceptable para un sistema de producción.

El manejo de fallos del Registry Service responde a la necesidad de mantener el servicio de descubrimiento disponible incluso cuando algunos nodos del Registry fallan. Si el líder del Registry falla, se realiza automáticamente una elección de líder, y el nuevo líder asume el catálogo de servicios. Los servicios del sistema continúan operando normalmente durante esta transición porque pueden consultar cualquier nodo del Registry, no solo el líder. Los clientes implementan failover automático entre múltiples nodos del Registry, permitiendo que continúen descubriendo servicios incluso si algunos nodos del Registry no responden. Esta capacidad de failover es esencial porque el Registry es un componente fundamental: sin acceso al Registry, los clientes no pueden descubrir otros servicios del sistema.

Nivel de Tolerancia a Fallos Esperado

El nivel de tolerancia a fallos que un sistema puede proporcionar está determinado por su arquitectura de replicación y los mecanismos de recuperación que implementa. El sistema TBFS está diseñado para tolerar fallos de múltiples componentes simultáneamente, proporcionando diferentes niveles de funcionalidad según qué componentes fallen.

La tolerancia a fallos del MetaNameNode está diseñada para permitir que el sistema continúe operando incluso cuando fallan hasta dos de los tres nodos. Con dos réplicas activas, el sistema puede continuar procesando operaciones de lectura y escritura, mantener consenso para operaciones de escritura mediante mayoría, y realizar elecciones de líder si es necesario. Esta capacidad de operar con mayoría proporciona alta disponibilidad: el sistema puede tolerar fallos de hasta un tercio de los nodos del MetaNameNode sin pérdida de funcionalidad. Con una sola réplica activa, el sistema puede continuar procesando operaciones, pero ya no puede lograr consenso ni realizar elecciones de líder debido a la ausencia de otros nodos. En este estado, el sistema proporciona servicio limitado: puede procesar operaciones de lectura y escritura, pero no puede garantizar consistencia fuerte ni recuperarse automáticamente de fallos adicionales.

La tolerancia a fallos de DataNodes está diseñada para permitir que el sistema continúe sirviendo archivos incluso cuando fallan hasta dos de los tres DataNodes que almacenan cada archivo. Con dos réplicas activas, el sistema puede continuar sirviendo solicitudes de lectura, re-replicar automáticamente archivos afectados a nuevos DataNodes, y mantener disponibilidad de todos los archivos. Esta capacidad proporciona alta disponibilidad para datos: incluso si un DataNode falla, todos los archivos permanecen accesibles desde las réplicas restantes.

Con una sola réplica activa, el sistema puede continuar sirviendo solicitudes de lectura, pero ya no puede re-replicar automáticamente archivos afectados debido a la ausencia de nodos adicionales. En este estado, el sistema proporciona acceso a datos pero con capacidad reducida de recuperación: si el DataNode restante falla, los archivos se perderían permanentemente.

La tolerancia a fallos del Registry está diseñada para permitir que el sistema continúe proporcionando descubrimiento de servicios incluso cuando fallan hasta dos de los tres nodos del Registry. Con dos réplicas activas, el Registry puede continuar proporcionando descubrimiento de servicios, mantener consenso para actualizaciones del catálogo mediante mayoría, y realizar elecciones de líder si es necesario. Esta capacidad proporciona alta disponibilidad del servicio de descubrimiento: el sistema puede tolerar fallos de hasta un tercio de los nodos del Registry sin pérdida de funcionalidad. Con una sola réplica activa, el Registry puede continuar proporcionando descubrimiento de servicios, pero ya no puede mantener consenso ni realizar elecciones de líder debido a la ausencia de otros nodos. En este estado, el Registry proporciona servicio limitado: puede responder a consultas de descubrimiento, pero no puede garantizar consistencia del catálogo ni recuperarse automáticamente de fallos adicionales.

Fallos Parciales

Los fallos parciales, donde algunos componentes fallan temporalmente y luego se recuperan, o donde nuevos componentes se incorporan al sistema, presentan desafíos adicionales que requieren mecanismos específicos de manejo.

Los nodos que fallan temporalmente y luego se recuperan requieren mecanismos que permitan su reintegración al sistema sin causar inconsistencias o interrupciones. Cuando un nodo se reinicia, se registra automáticamente con el sistema, indicando que está nuevamente disponible. El sistema detecta esta reintegración y actualiza su estado interno para reflejar que el nodo está activo. Para DataNodes que se recuperan, puede haber una situación donde los archivos que estaban almacenados en el DataNode fueron re-replicados durante su ausencia. El sistema maneja esta situación detectando archivos duplicados y puede limpiarlos si es necesario, o puede mantener las réplicas adicionales temporalmente para proporcionar mayor redundancia. La reintegración automática sin intervención manual es esencial para mantener la operación del sistema: en un entorno de producción, los nodos pueden fallar y recuperarse frecuentemente debido a mantenimiento, actualizaciones, o fallos temporales de hardware.

Los nodos nuevos que se incorporan al sistema requieren mecanismos que permitan su integración gradual sin interrumpir las operaciones existentes. Cuando un nuevo DataNode se inicia, se registra automáticamente con el MetaNameNode, proporcionando información sobre su capacidad y estado. El MetaNameNode entonces comienza a asignar nuevas réplicas a este DataNode, integrando gradualmente el nuevo nodo en la distribución de carga. Esta integración gradual es importante porque permite que el sistema aproveche nueva capacidad sin re-

querir redistribución masiva de datos existentes, lo cual sería costoso y podría interrumpir el servicio. La capacidad de incorporar nuevos nodos dinámicamente sin reiniciar el sistema es crucial para la escalabilidad: permite que el sistema crezca para manejar cargas crecientes sin interrupciones.

El drenaje controlado de nodos proporciona un mecanismo para remover DataNodes del sistema de forma segura sin pérdida de datos. Cuando un DataNode necesita ser removido (por ejemplo, para mantenimiento o reemplazo), se marca para drenaje, lo que evita que se le asignen nuevos archivos. El sistema entonces re-replica todos los archivos almacenados en el DataNode a otros DataNodes disponibles. Una vez que todos los archivos han sido re-replicados, el DataNode puede ser removido del sistema sin pérdida de datos. Este proceso de drenaje permite mantenimiento planificado sin interrupciones: los administradores pueden programar mantenimiento de hardware o actualizaciones de software sin afectar la disponibilidad del sistema. La capacidad de drenar nodos de forma controlada es esencial para la mantenibilidad del sistema en entornos de producción.

La recuperación automática proporciona mecanismos que restauran el sistema a su estado normal después de fallos sin requerir intervención manual. Los procesos de re-replicación se ejecutan automáticamente en segundo plano cuando se detectan fallos, restaurando el nivel de replicación deseado. El sistema restaura automáticamente la capacidad de tolerar fallos adicionales mediante la re-replicación de datos afectados. Los clientes experimentan mínima interrupción durante la recuperación porque el sistema puede continuar sirviendo datos desde réplicas activas mientras la recuperación ocurre. La capacidad de recuperación automática sin intervención manual es esencial para la operación del sistema en entornos de producción donde los fallos ocurren regularmente y la intervención manual inmediata no siempre es posible.

8. Seguridad: El Problema de Qué Tan Vulnerable es el Diseño

Seguridad con Respecto a la Comunicación

La seguridad en la comunicación es fundamental en cualquier sistema distribuido, ya que los datos y comandos viajan a través de redes que pueden ser interceptadas o modificadas por actores maliciosos. El sistema TBFS implementa varias estrategias para proteger las comunicaciones, aunque reconoce que la seguridad completa requiere consideraciones adicionales para entornos de producción.

El protocolo de comunicación actual utiliza HTTP para todas las comunicaciones, proporcionando una base funcional pero sin cifrado inherente. Esta elección responde a la necesidad de simplicidad en desarrollo y despliegue, pero reconoce que para entornos de producción se requeriría migración a HTTPS. HTTPS proporcionaría cifrado de todas las comunicaciones mediante TLS, garantizando que los datos transmitidos no puedan ser leídos por terceros.

que intercepten el tráfico de red. Además, HTTPS proporciona autenticación mediante certificados, permitiendo que los servicios verifiquen la identidad de otros servicios con los que se comunican. Esta autenticación es crucial para prevenir ataques donde un actor malicioso se hace pasar por un servicio legítimo, lo cual podría resultar en redirección de tráfico hacia servicios comprometidos o modificación de datos en tránsito.

La comunicación en red privada mediante la red overlay de Docker Swarm proporciona una capa adicional de seguridad mediante aislamiento de red. Los servicios se comunican a través de una red virtual privada que no es accesible desde fuera del swarm, proporcionando protección contra acceso no autorizado desde la red pública. Este aislamiento es fundamental porque limita la superficie de ataque del sistema: incluso si un servicio tiene vulnerabilidades, estas solo pueden ser explotadas por componentes que tienen acceso a la red privada. La separación entre comunicación interna (usando la red overlay) y acceso externo (mediante puertos expuestos específicos) permite que el sistema controle qué servicios son accesibles desde fuera del swarm, proporcionando un modelo de seguridad de mínimo privilegio donde cada servicio solo expone lo que es necesario.

La validación de datos en todas las comunicaciones proporciona protección contra ataques de inyección y corrupción de datos. El sistema implementa validación estricta de tipos y formatos de datos mediante modelos de datos estructurados, garantizando que los datos recibidos cumplan con las expectativas del sistema antes de procesarlos. Esta validación previene errores que podrían explotarse para causar comportamientos inesperados o fallos del sistema. La sanitización de entradas previene ataques donde datos maliciosos podrían explotar vulnerabilidades en el procesamiento de datos, como inyección de código o desbordamientos de buffer. La validación de tamaños de archivo y límites de recursos previene ataques de denegación de servicio donde actores maliciosos intentan consumir todos los recursos del sistema mediante solicitudes excesivamente grandes o numerosas.

Seguridad con Respecto al Diseño

La seguridad en el diseño se refiere a cómo la arquitectura del sistema limita el impacto de compromisos potenciales y previene la propagación de vulnerabilidades. El sistema TBFS implementa varios principios de diseño que proporcionan seguridad inherente mediante la estructura del sistema.

La separación de responsabilidades entre MetaNameNode y DataNodes proporciona una barrera arquitectónica que limita el alcance de un compromiso potencial. Si un DataNode es comprometido, el atacante solo tiene acceso a los archivos almacenados en ese DataNode, pero no puede acceder directamente a los metadatos del sistema ni modificar la asignación de réplicas. Esta limitación es crucial porque previene que un compromiso de un componente de almacenamiento se propague a la gestión de metadatos, lo cual podría resultar

en pérdida o corrupción de información sobre todos los archivos del sistema. Inversamente, si el MetaNameNode es comprometido, el atacante tiene acceso a información sobre archivos y sus ubicaciones, pero no puede acceder directamente al contenido de los archivos físicos sin también comprometer los DataNodes correspondientes. Esta separación de responsabilidades crea múltiples capas de defensa donde un compromiso de un componente no resulta automáticamente en compromiso completo del sistema.

El aislamiento de contenedores mediante Docker proporciona límites de seguridad a nivel de sistema operativo. Cada servicio se ejecuta en un contenedor aislado con límites de recursos (CPU, memoria) que previenen que un servicio comprometido consuma todos los recursos del sistema. El aislamiento de sistemas de archivos previene que servicios accedan a archivos de otros servicios, incluso si un servicio es comprometido. Este aislamiento es fundamental porque previene que vulnerabilidades en un servicio se propaguen a otros servicios mediante acceso directo a archivos o recursos compartidos. En un entorno de Docker Swarm, este aislamiento se extiende a través de múltiples nodos físicos, proporcionando protección incluso cuando servicios se ejecutan en diferentes máquinas.

La gestión de volúmenes persistentes con aislamiento por servicio proporciona protección adicional para datos persistentes. Cada MetaNameNode tiene su propio volumen de base de datos que no es accesible por otros servicios, previniendo que un servicio comprometido acceda a datos de otros servicios. Cada DataNode tiene su propio volumen de almacenamiento que solo es accesible por ese DataNode específico, previniendo acceso cruzado entre volúmenes. Este aislamiento de volúmenes es crucial porque los datos persistentes contienen información crítica que debe protegerse incluso si los contenedores son comprometidos o recreados. La prevención de acceso cruzado entre volúmenes garantiza que un compromiso de un servicio no resulte en acceso a datos de otros servicios.

La validación de integridad mediante hashes criptográficos proporciona protección contra corrupción de datos, ya sea accidental o maliciosa. Cada archivo se identifica mediante un hash SHA-256 de su contenido, proporcionando una verificación inherente de integridad. Cuando se lee un archivo, el sistema puede verificar que el contenido no ha sido modificado comparando el hash calculado del contenido leído con el hash almacenado en los metadatos. Esta verificación de integridad es esencial porque detecta corrupción de datos que podría ocurrir debido a fallos de hardware, errores de software, o modificaciones maliciosas. La capacidad de detectar corrupción permite que el sistema intente recuperarse leyendo desde otras réplicas, previniendo que datos corruptos se propaguen o se sirvan a clientes.

Autorización y Autenticación

La autorización y autenticación son aspectos críticos de seguridad que determinan quién puede acceder al sistema y qué operaciones pueden realizar. El

sistema TBFS actualmente no implementa autenticación ni autorización, una decisión que responde a su diseño para entornos de desarrollo o redes privadas confiables, pero que requeriría extensión para entornos de producción.

El estado actual del sistema asume que todos los componentes operan en un entorno de confianza, donde no hay necesidad de verificar la identidad de clientes o servicios antes de permitir operaciones. Esta suposición simplifica significativamente el diseño y la implementación del sistema, permitiendo que se enfoque en los problemas fundamentales de distribución, replicación y tolerancia a fallos sin la complejidad adicional de sistemas de autenticación y autorización. Esta simplificación es apropiada para entornos de desarrollo, pruebas, o redes privadas donde todos los componentes son controlados por la misma organización y se asume que no hay actores maliciosos.

Para entornos de producción, se requerirían extensiones significativas al sistema de seguridad. La autenticación de clientes mediante sistemas basados en tokens (como JWT) permitiría que el sistema verifique la identidad de usuarios antes de permitir operaciones. Esta autenticación requeriría que los clientes proporcionen credenciales válidas y recibieran tokens que demuestren su identidad en solicitudes posteriores. La validación de credenciales antes de permitir operaciones previene acceso no autorizado al sistema, mientras que la gestión de sesiones permite que los clientes mantengan su autenticación a través de múltiples solicitudes sin requerir credenciales en cada solicitud.

La autorización basada en roles permitiría que el sistema controle qué operaciones puede realizar cada usuario según su rol o permisos. Un sistema de control de acceso basado en roles (RBAC) permitiría definir diferentes niveles de acceso, como usuarios que solo pueden leer archivos, usuarios que pueden leer y escribir, y administradores que pueden realizar operaciones de gestión del sistema. Los permisos diferenciados para lectura y escritura permitirían que el sistema implemente políticas donde algunos usuarios pueden leer archivos pero no modificarlos, proporcionando protección contra modificaciones no autorizadas. La auditoría de operaciones proporcionaría trazabilidad, permitiendo que el sistema registre quién realizó qué operaciones y cuándo, lo cual es esencial para detectar accesos no autorizados o investigar incidentes de seguridad.

La autenticación entre servicios mediante autenticación mutua TLS (mTLS) permitiría que los servicios verifiquen la identidad de otros servicios antes de aceptar comunicaciones. Esta autenticación mutua es crucial en un sistema distribuido donde los servicios se comunican entre sí, porque previene ataques donde un actor malicioso se hace pasar por un servicio legítimo. Los tokens de servicio proporcionarían una alternativa o complemento a mTLS, permitiendo que los servicios demuestren su identidad mediante tokens que son verificados por servicios receptores. La validación de identidad de servicios antes de aceptar peticiones previene ataques donde servicios maliciosos intentan inyectar datos falsos o redirigir tráfico hacia servicios comprometidos.

La gestión de secretos requeriría sistemas especializados para almacenar y ges-

tionar credenciales, claves de cifrado, y otros secretos de forma segura. El almacenamiento seguro de credenciales y claves es esencial porque la exposición de estos secretos resultaría en compromiso completo del sistema. La rotación periódica de secretos limitaría el impacto de una exposición de secretos, ya que los secretos expuestos eventualmente expirarían y serían reemplazados. El uso de sistemas de gestión de secretos especializados (como HashiCorp Vault) proporcionaría capacidades avanzadas como cifrado de secretos en reposo, control de acceso granular a secretos, y auditoría de acceso a secretos.

La limitación de acceso mediante firewalls y reglas de red proporcionaría una capa adicional de seguridad mediante restricción de qué componentes pueden comunicarse entre sí. Las reglas de firewall podrían restringir el tráfico de red a solo lo que es necesario para el funcionamiento del sistema, previniendo comunicación no autorizada entre componentes. El whitelisting de IPs permitidas limitaría el acceso al sistema a solo direcciones IP conocidas y confiables, previniendo acceso desde ubicaciones no autorizadas. El rate limiting prevendría ataques de denegación de servicio mediante limitación de la frecuencia con la que se pueden realizar solicitudes, previniendo que actores maliciosos sobrecarguen el sistema con solicitudes excesivas.